

CSE / EEE / ETE 499A (Section 1)

Design Report (CO2)

Project Title:

ShopMate AI — A Catalog-Grounded, Omnichannel AI Sales & Support Agent for Facebook, Instagram, WhatsApp, and Website Commerce

Submitted To

Dr. Shazzad Hosain (SZZ)

Date: 19/07/2026



Semester: Summer 2026

Group No: G-1

Group Members

ID	Name
2311966642	Safat Uddin
2311790642	Farhan Karim
2311443042	Safwan Ismaun Amin
2311195042	Mahir Shahriyar Adib

1. Use Case Diagram

The use case diagram below captures the three internal actors (Customer, Merchant, System Admin) and the three external systems the platform integrates with (Meta/WhatsApp platform APIs, the merchant's e-commerce backend, and a payment gateway). <<extend>> relationships show that receiving an AI reply is an extension of sending a message, and that a human-handled complaint is an extension of the base “report a problem” use case — i.e., the system always attempts an automatic response first and only escalates to a human when the complaint classifier or the grounding guardrail flags it (Section 3c).

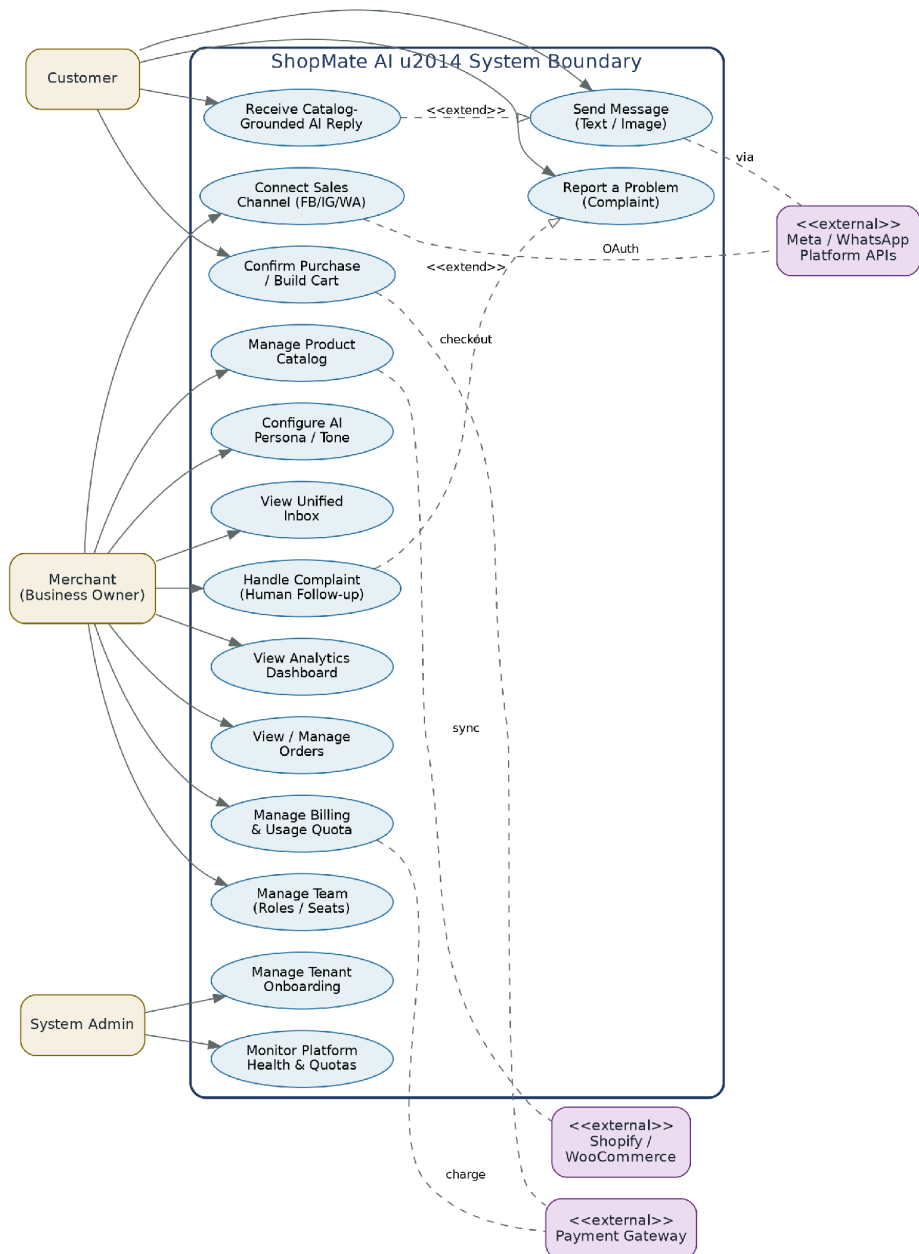


Figure 1.1 — Use Case Diagram for ShopMate AI

1.1 Actor and Use Case Summary

Actor	Type	Primary Use Cases
Customer	Primary	Send message (text/image), receive catalog-grounded AI reply, confirm purchase / build cart, report a problem.
Merchant	Primary	Connect a sales channel, manage product catalog, configure AI persona/tone, view unified inbox, handle escalated complaints, view analytics, manage orders, manage billing/quota, manage team.
System Admin	Primary	Monitor platform health & quotas across tenants, manage tenant onboarding/offboarding.
Meta / WhatsApp Platform APIs	External	Delivers inbound webhooks; receives outbound sends; brokers channel OAuth connection.
Shopify / WooCommerce	External	Source of truth for product catalog sync.
Payment Gateway	External	Processes checkout and subscription billing charges.

2. UX Designs

The interface follows a single design language across all authenticated screens: a persistent dark left-navigation rail (Unified Inbox, Product Catalog, Analytics, Integrations, Settings, Support), a consistent top bar (contextual search, notifications, a quick-action bolt icon, and account avatar), and a “command center” visual identity — dark background, high-contrast white typography, and a single neutral accent — chosen deliberately to position the product as a premium, high-precision tool rather than a generic chatbot builder, directly supporting the “competitive advantage” claims in the CO1 proposal (Section 3.4).

A second recurring pattern is **AI transparency**: every AI-authored artifact in the interface is explicitly labelled rather than presented as if a human wrote it — the inbox marks drafts as “AI-Generated Reply” with a visible “Drafted just now” timestamp and requires a merchant to press Send Now before anything reaches the customer; the catalog page shows a per-product “AI Status” (Trained / Pending); and the AI Strategy panel exposes its own confidence score (e.g., “Probability of conversion: 92%”) rather than issuing an unexplained recommendation. This directly operationalizes the hard grounding rule from Section 10.3 of the architecture blueprint at the UI layer: the merchant always has a human-in-the-loop checkpoint before an AI-drafted claim reaches a customer.

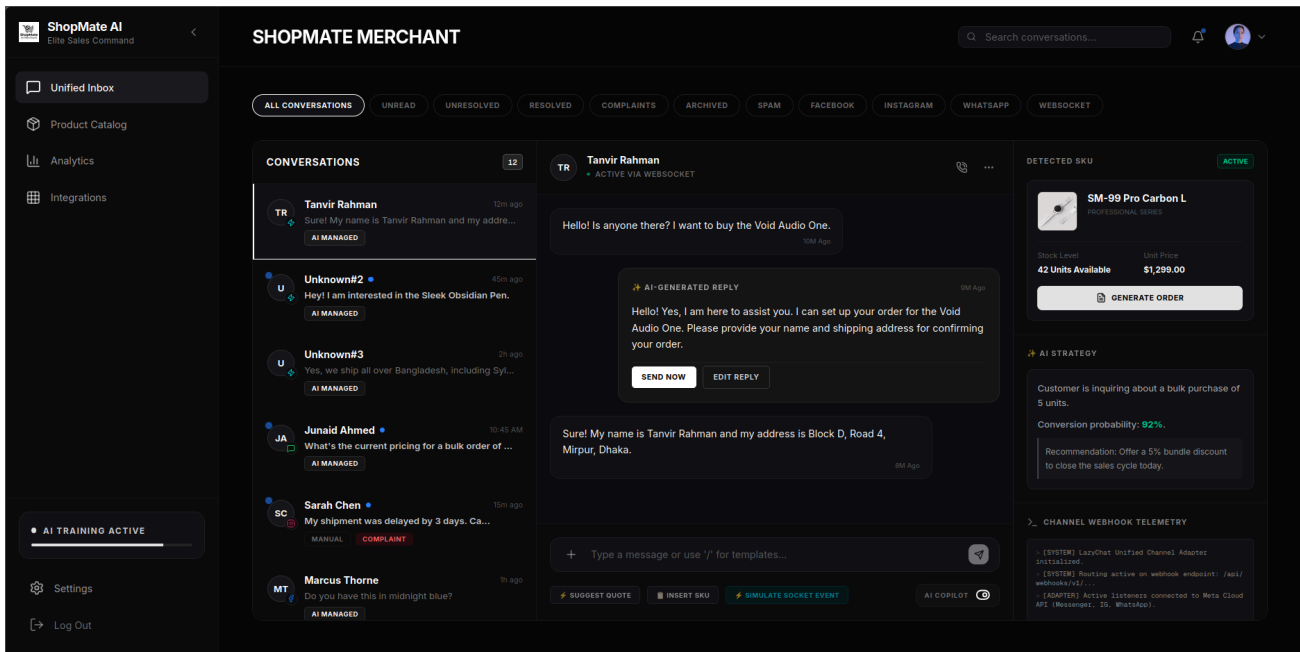


Figure 2.1 — Unified Inbox Dashboard(implemented high-fidelity mockup)

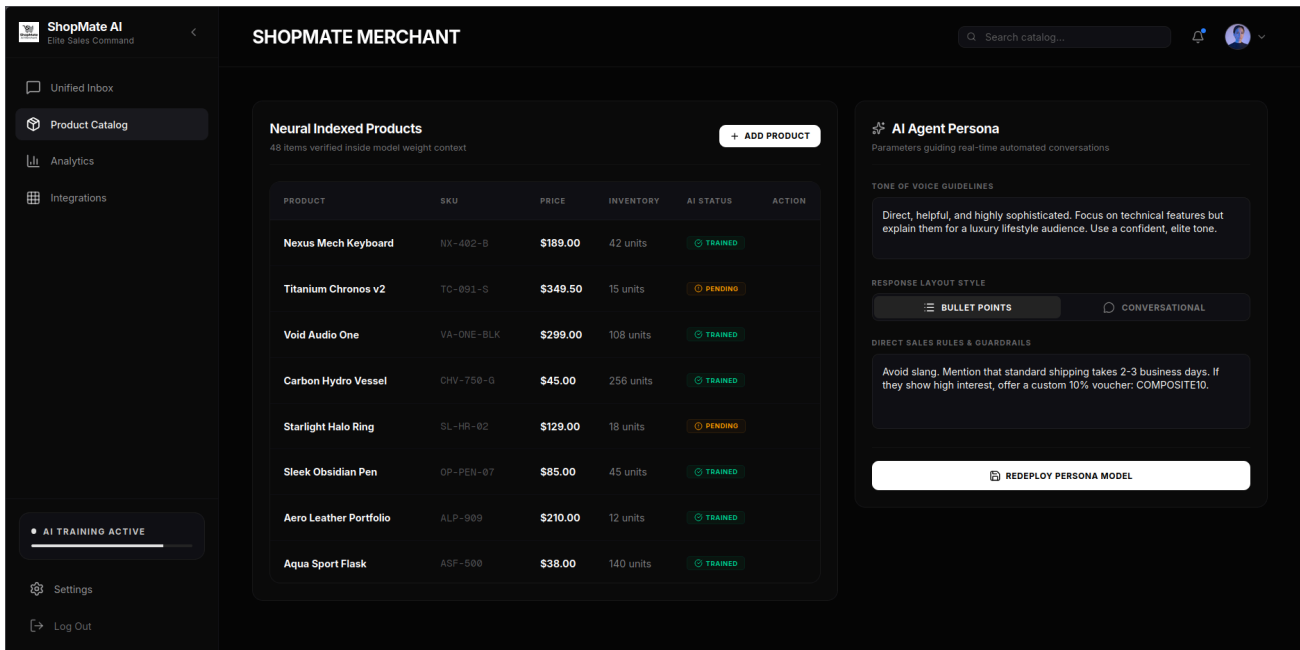


Figure 2.2 — Product Catalog Dashboard(implemented high-fidelity mockup)

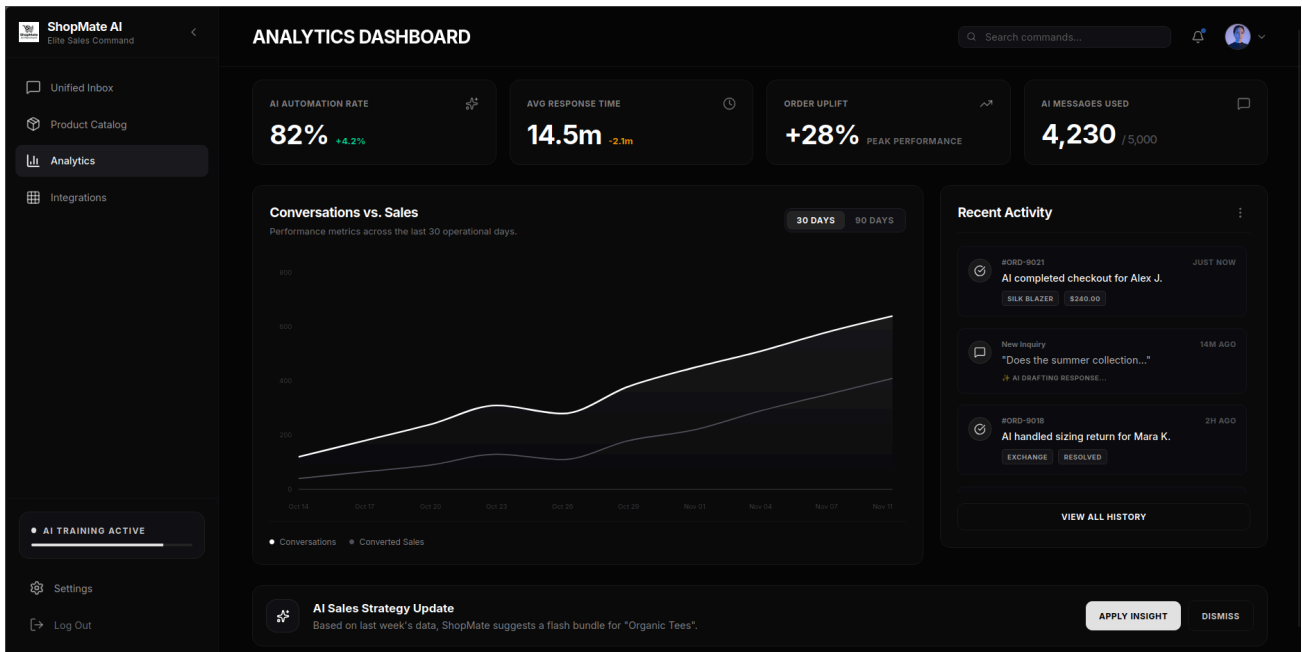


Figure 2.3 — Analytics Dashboard (implemented high-fidelity mockup)

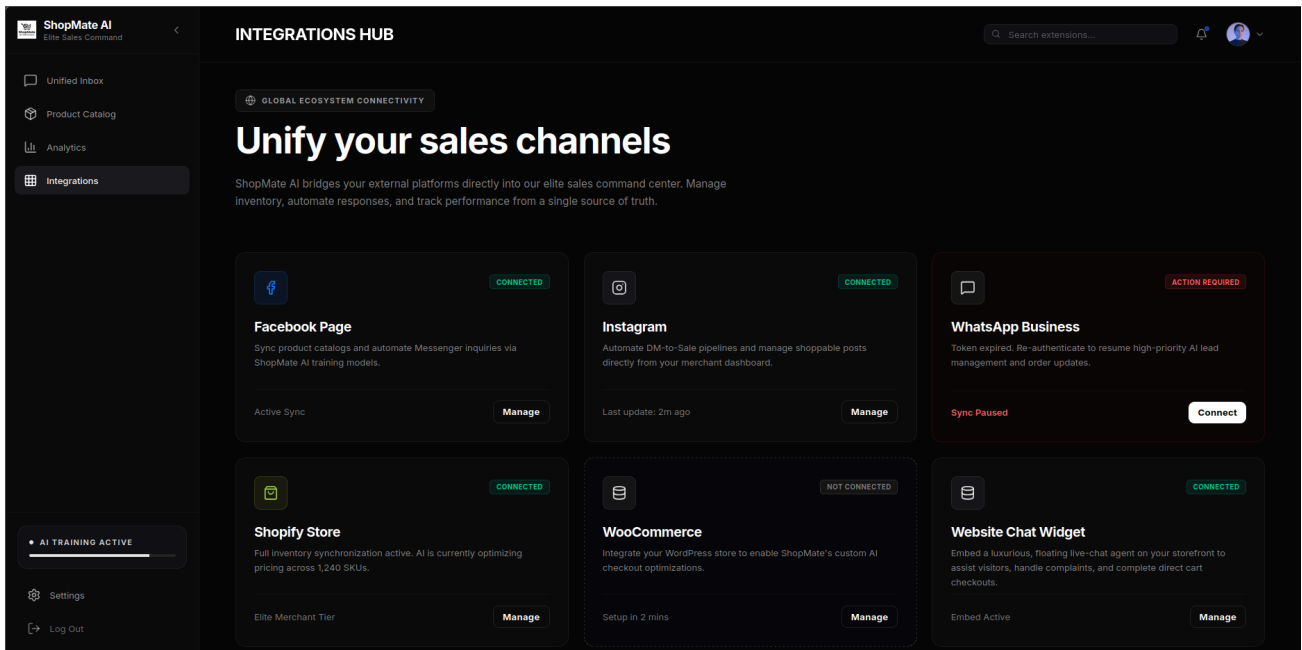


Figure 2.4 — Facebook, Instagram, Whatsapp & Shopify, WooCommerce Integration Hub Dashboard

Figure 2.5 — Landing Page

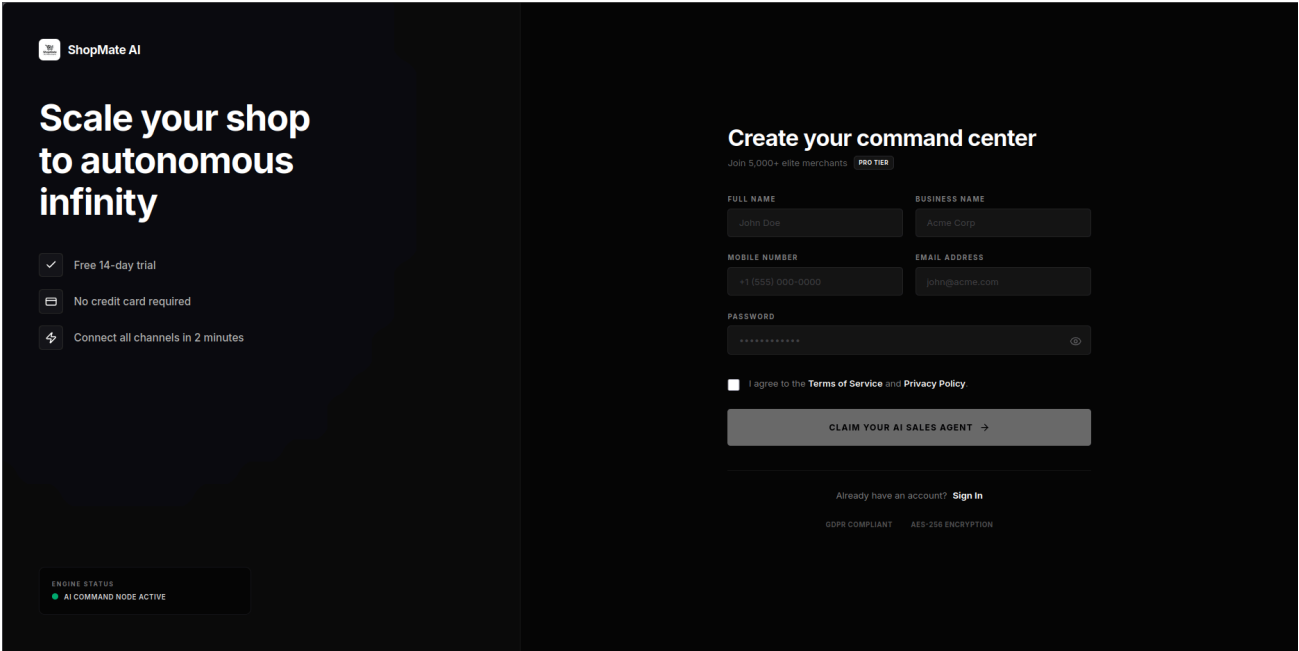


Figure 2.5 — Sign-up Page

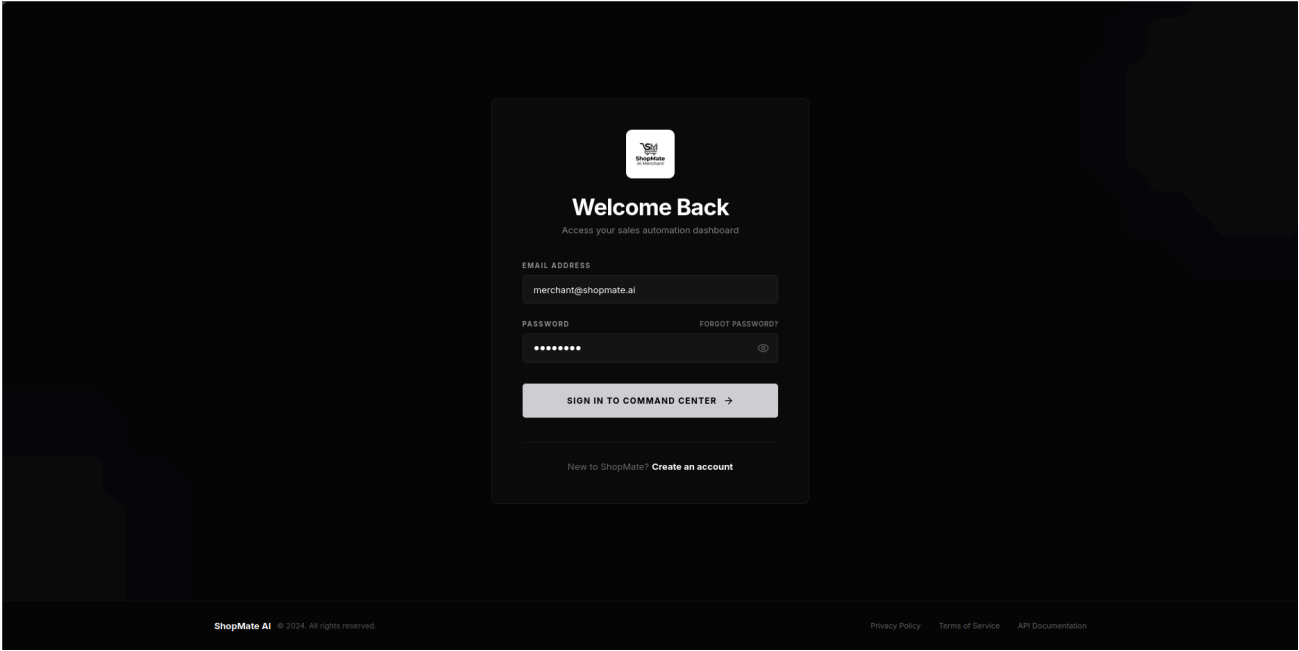


Figure 2.7 — Sign-in Page

2.1 Screen Inventory

Screen	Purpose	Key UI Elements
Marketing / Landing Page	Convert visiting merchants into trial sign-ups; establish the “elite, precision-engineered” brand positioning.	Hero headline (“Turn Every Message into a Sale”) with a live conversation preview widget; proof stats (98% inquiry accuracy, 3.2x conversion, 24/7 uptime, 15+ languages); feature grid (Visual Intelligence, Always On / Bangla-Banglish, One-Click Checkout, Unified Command); local retail-brand logo strip; dual CTA footer.
Sign Up	Low-friction merchant onboarding.	Split layout: left panel reinforces value proposition (14-day trial, no card required, 2-minute setup) with a live “Engine Status” indicator; right panel is a single-column form (name, business name, mobile, email, password) with a Terms checkbox and GDPR/AES-256 trust badges.
Sign In	Returning-merchant authentication.	Centered card, brand mark, email/password fields, “Forgot Password” link, primary CTA, Google/Facebook social sign-in, and a link back to Sign Up.
Unified Inbox	Single pane of glass for every channel; the system's core daily-use screen.	Left navigation rail; channel/status filter tabs (All, Unresolved, Complaints, Facebook, Instagram, WhatsApp, Webhook); conversation list with intent/complaint tag badges; center thread view with a clearly labelled “AI-Generated Reply” block offering Send Now / Edit Reply; right-hand “Detected SKU” card with a one-click “Generate Order” action and an “AI Strategy” recommendation panel.
Product Catalog	Catalog visibility and AI-persona configuration — the two levers that most directly control reply quality.	Sync-status stat cards (Total Products, AI Trained, Pending Sync, Training Status progress bar); a paginated product table with per-item AI training status; a right-hand “AI Persona Settings” panel exposing the system prompt (tone of voice, response style) merchants can edit without touching code.
Integrations Hub	Central control panel for every external connector.	Card grid per channel (Facebook, Instagram, WhatsApp, Shopify, WooCommerce) showing Connected / Action-Required / Not-Connected state; a WhatsApp token-expiry warning demonstrates how the UI surfaces the platform-policy risk noted in Section 3.6 of the proposal; an AI-generated “Integration Insight” upsell suggestion.
Analytics Dashboard	Proves ROI to the merchant and surfaces operational alerts.	KPI cards (AI Automation Rate, Avg Response Time, Order Uplift, AI Messages Used vs. quota); a Conversations-vs-Sales trend chart; a Recent Activity feed (completed AI checkouts, resolved exchanges, low-stock alerts); an actionable “AI Sales Strategy Update” banner.

2.2 Key Interaction Flow

The most business-critical flow is inside the Unified Inbox: a customer message arrives → the AI drafts a catalog-grounded reply and attaches a “Detected SKU” card when a product is identified → the merchant reviews and either sends the draft as-is or edits it → on purchase intent, the merchant (or the AI, per the tenant's automation-level setting) presses “Generate Order” to convert the conversation directly into an order record, closing the loop between chat and commerce without the merchant leaving the inbox. This flow is expanded into a full sequence diagram in Section 5.2.

3. System Designs

3.1 System Architectural Design

The system is organized into seven layers, from channel-facing edge adapters down to the merchant-facing dashboard, communicating primarily through an asynchronous message bus so that a spike on one channel (e.g., a flash-sale flood of WhatsApp messages) cannot block another channel's ingestion or the dashboard's read path.

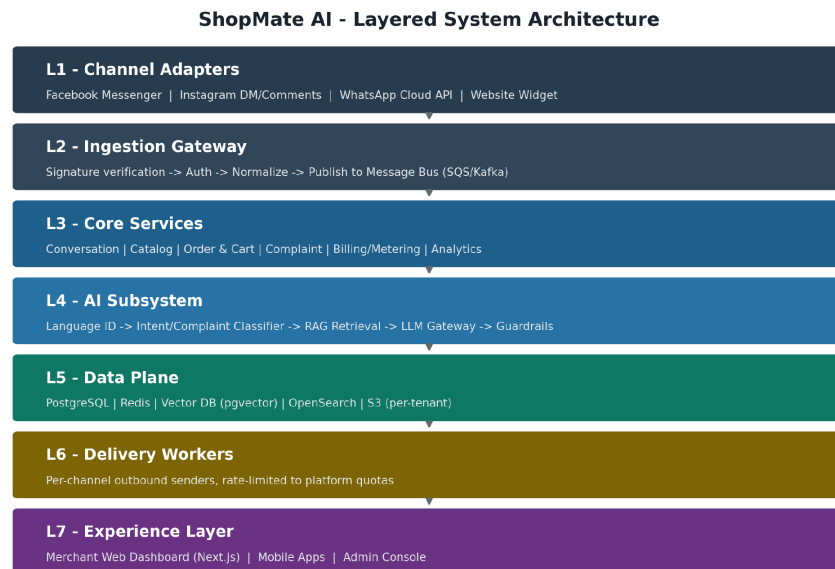


Figure 3.1 — Layered System Architecture

- **Edge / Channel Adapters** — one adapter per external platform (Messenger, Instagram, WhatsApp Cloud API, website widget), each responsible only for translating a platform-specific payload into, and out of, the system's internal message format.
- **Ingestion Gateway** — verifies webhook signatures, deduplicates by provider message ID (idempotency), and publishes a normalized event onto the message bus; this is the system's single trust boundary between untrusted internet traffic and internal services.
- **Core Services** — Conversation, Catalog, Order & Cart, Complaint, Billing/Metering, and Analytics — independently deployable services, each owning its own data.
- **AI Subsystem** — language detection, intent/complaint classification, RAG retrieval, LLM Gateway, and a grounding guardrail (detailed in Section 3.2).
- **Data Plane** — PostgreSQL for transactional data, Redis for cache/session state, pgvector for product embeddings, OpenSearch for full-text/analytics queries, and S3 for per-tenant media, all partitioned per tenant.
- **Delivery Workers** — per-channel outbound senders that respect each platform's rate limits and messaging-window rules (e.g., WhatsApp's 24-hour customer-service window).
- **Experience Layer** — the merchant web dashboard shown in Section 2, built as the primary client of the Core Services' APIs.

3.2 Sub-Component Design

The AI Subsystem is the most novel part of the system and is decomposed further below, since it is where the project's central design constraint — never state an ungrounded price, stock level, or promise — is actually enforced in code rather than left to a prompt instruction.

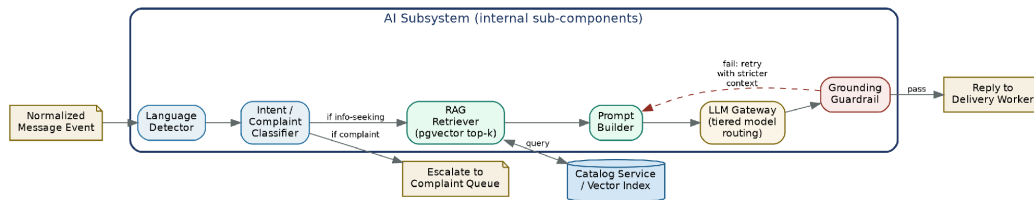


Figure 3.2 — AI Subsystem Internal Sub-Components

A normalized message enters the Language Detector, then the Intent/Complaint Classifier. If the classifier flags the message as a complaint, it is routed directly to the human-follow-up queue and never reaches the LLM (avoiding the risk of an AI-authored response to an already-upset customer). Otherwise, the RAG Retriever embeds the query and performs a top-

k similarity search against the tenant's product-embedding index in the Catalog Service; the retrieved passages are assembled by the Prompt Builder into a context-constrained prompt; the LLM Gateway (Section 3.3) generates a draft; and the Grounding Guardrail checks every factual claim in the draft against the retrieved context before it is allowed to reach the Delivery Worker. A failed guardrail check triggers one bounded retry with a stricter, narrower context rather than silently returning the ungrounded draft.

Other sub-components of note

- **Ingestion Gateway** — signature verifier, idempotency-key store (Redis, 24h TTL), and a per-tenant token-bucket rate limiter (Section 6.4).
- **Catalog Service** — source connectors (Shopify/WooCommerce), an embedding pipeline that re-embeds a product only when its title/description/price/stock changes (change-data-capture, not a fixed schedule), and a read-through cache for hot SKUs.
- **Order & Cart Service** — a small state machine (draft → confirmed → fulfilled → cancelled) that the “Generate Order” action in the inbox drives, and that is the single writer of the orders/order_items tables.
- **Metering & Billing Service** — appends one usage_events row per AI-generated message and periodically aggregates it against the tenant's subscription quota (Section 6.5).

3.3 System Design and Implementation Challenges

Challenge	Why it is hard	How it is addressed
Hallucinated price / stock answers	LLMs will confidently state a plausible-sounding but wrong number if not constrained.	Grounding Guardrail (Section 3.2) rejects any generated numeric claim not traceable to a retrieved passage; on failure the system retries once with a narrower prompt, then falls back to a safe “let me check and get back to you” template and escalates to the merchant.
Bangla / English / Banglish in one thread	Standard NLU pipelines are trained overwhelmingly on monolingual English text and misclassify code-mixed input.	A dedicated language-ID step runs before intent classification (not a single combined model), and the LLM is prompted to reply in the customer's detected register; this follows the SentMix-3L finding that LLM-based zero/few-shot handling outperforms bespoke fine-tuned classifiers on Bangla-English-mixed text (CO1 Reference 6).
Bursty, duplicate webhooks	Meta and WhatsApp retry undelivered webhooks automatically, and a flash sale can produce thousands of near-simultaneous events.	Every inbound webhook is deduplicated by provider message ID at the Ingestion Gateway before it is published to the bus, and all downstream consumers are written to be idempotent (safe to process the same event twice).
Multi-tenant data isolation	A single shared database serving many merchants risks one tenant's data leaking into another's AI context or dashboard.	Every table carries a tenant_id and every query path enforces PostgreSQL row-level security; the RAG Retriever's vector search is always pre-filtered by tenant_id so cross-tenant retrieval is structurally impossible, not just policy-forbidden.
LLM cost control	Naively calling a frontier model for every classification step would make per-message cost unpredictable and high.	Tiered model routing (Section 6.6): cheap/fast models handle language ID, intent, and complaint classification; a stronger model is reserved for the final customer-facing generation step only.
Platform policy compliance (WhatsApp 24h window, Meta business verification)	Sending outside the allowed messaging window or with an unverified app can get a channel suspended.	The Delivery Worker checks the conversation's last-customer-message timestamp before every send and routes to an approved template message if the window has closed; the Integrations Hub UI (Section 2) surfaces token-expiry/re-authentication needs proactively.

4. Database Design / Hardware Component Description

This project is a pure software SaaS system with no custom hardware component; the “Hardware Component Description” branch of this section is therefore not applicable, and the section focuses on database design.

4.1 Entity-Relationship Diagram

The schema is intentionally tenant-first: almost every table carries a tenant_id foreign key so that row-level security policies (Section 3.3) can be enforced uniformly rather than re-implemented per table.

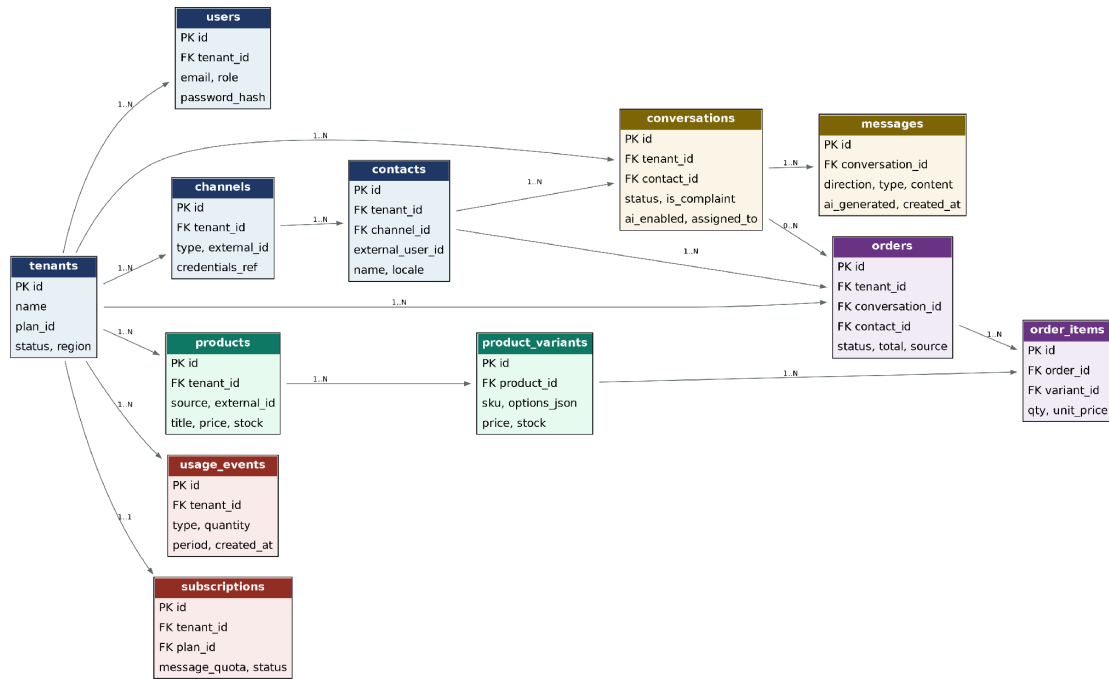


Figure 4.1 — Entity-Relationship Diagram

4.2 Schema Dictionary

Table	Purpose	Key Fields
tenants	One row per merchant account; the root of multi-tenant isolation.	id (PK), name, plan_id, status, region
users	Merchant-side dashboard users (owner, agent, admin roles) belonging to a tenant.	id (PK), tenant_id (FK), email, role, password_hash
channels	A connected external channel (Facebook Page, IG account, WhatsApp number, website widget) per tenant.	id (PK), tenant_id (FK), type, external_id, credentials_ref
contacts	A customer identity as seen on a specific channel.	id (PK), tenant_id (FK), channel_id (FK), external_user_id, name, locale
conversations	One thread per contact; the unit the inbox and AI orchestrator operate on.	id (PK), tenant_id (FK), contact_id (FK), status, is_complaint, ai_enabled, assigned_to
messages	Every inbound/outbound message in a conversation, including AI drafts.	id (PK), conversation_id (FK), direction, type, content, language, ai_generated, created_at
products / product_variants	The synced catalog and its purchasable variants (size/color/SKU-level).	id (PK), tenant_id (FK), source, external_id, title, price, stock; variant: sku, options_json
orders / order_items	Orders created from a conversation, and their line items.	id (PK), tenant_id (FK), conversation_id (FK), contact_id (FK), status, total; item: order_id (FK), variant_id (FK), qty, unit_price
usage_events	One row per billable AI action, feeding the Metering Service.	id (PK), tenant_id (FK), type, quantity, period, created_at
subscriptions	The tenant's current plan and message quota.	id (PK), tenant_id (FK), plan_id (FK), message_quota, status

4.3 Design Notes

- **Vector storage** — product embeddings are stored via the pgvector extension directly alongside the relational product row, avoiding a separate vector-database service at prototype scale while still supporting the RAG Retriever's top-k similarity query.
- **Append-only usage** — usage_events is intentionally append-only (never updated in place) so that billing calculations are auditable and reproducible.
- **Soft multi-tenancy boundary** — a shared schema with row-level security was chosen over one database per tenant, trading perfect physical isolation for lower operational overhead, appropriate for the SME-scale tenants this system targets in its first year.

5. Class Diagram, Sequence Diagram, etc.

5.1 Class Diagram

The class diagram mirrors the sub-component design in Section 3.2: ChannelAdapter is an abstract type realized by MessengerAdapter and WhatsAppAdapter (and, by the same pattern, an Instagram and website-widget adapter); AIOrchestrator composes the RAGRetriever and ComplaintClassifier rather than implementing retrieval or classification logic itself, keeping each concern independently testable.

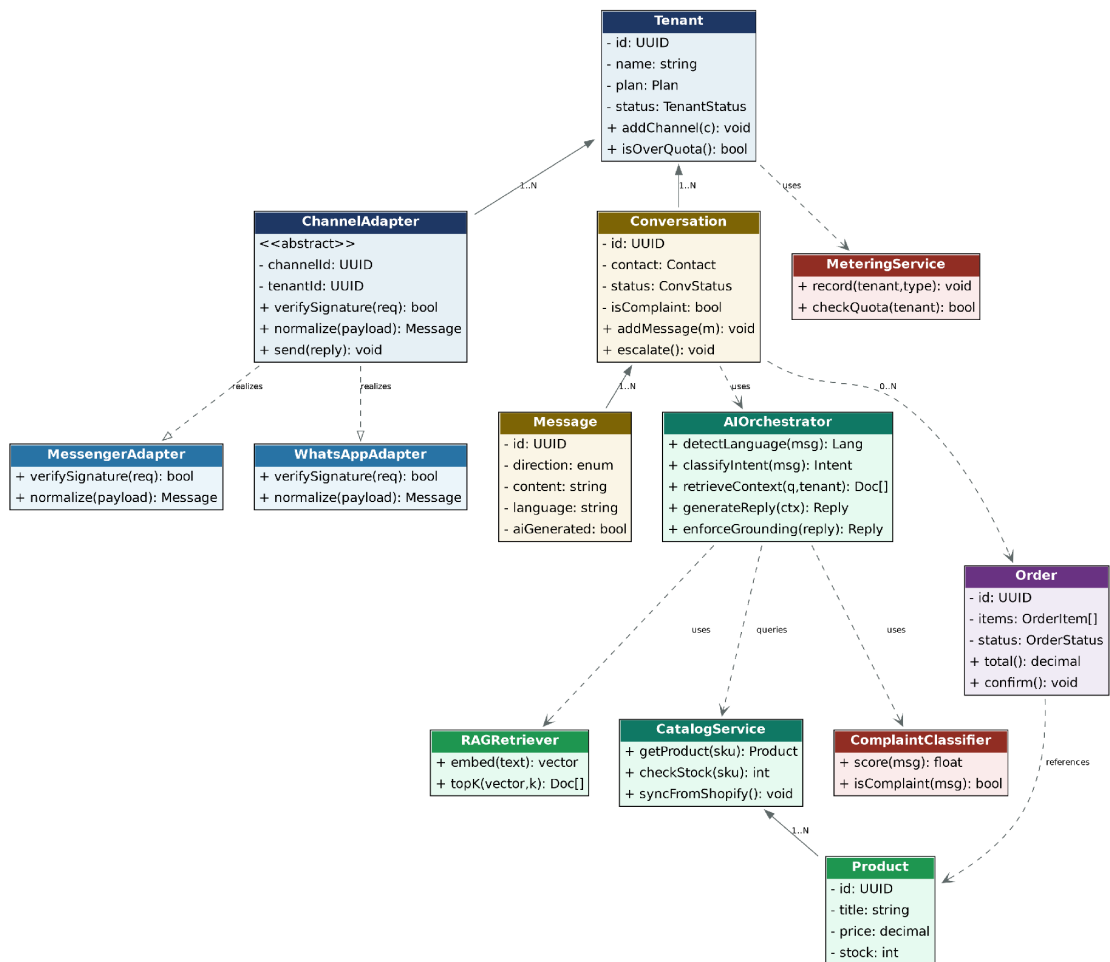


Figure 5.1 — Class Diagram (core domain and AI subsystem)

5.2 Sequence Diagram — Catalog-Grounded AI Reply Flow

The sequence diagram traces a single customer message from arrival to delivery, including the two decision points that most directly implement the project's risk mitigations from the CO1 proposal: the self-call at the AI Orchestrator that enforces the grounding rule (step 13), and the conditional order-creation branch (step 14) that only fires on detected purchase intent.

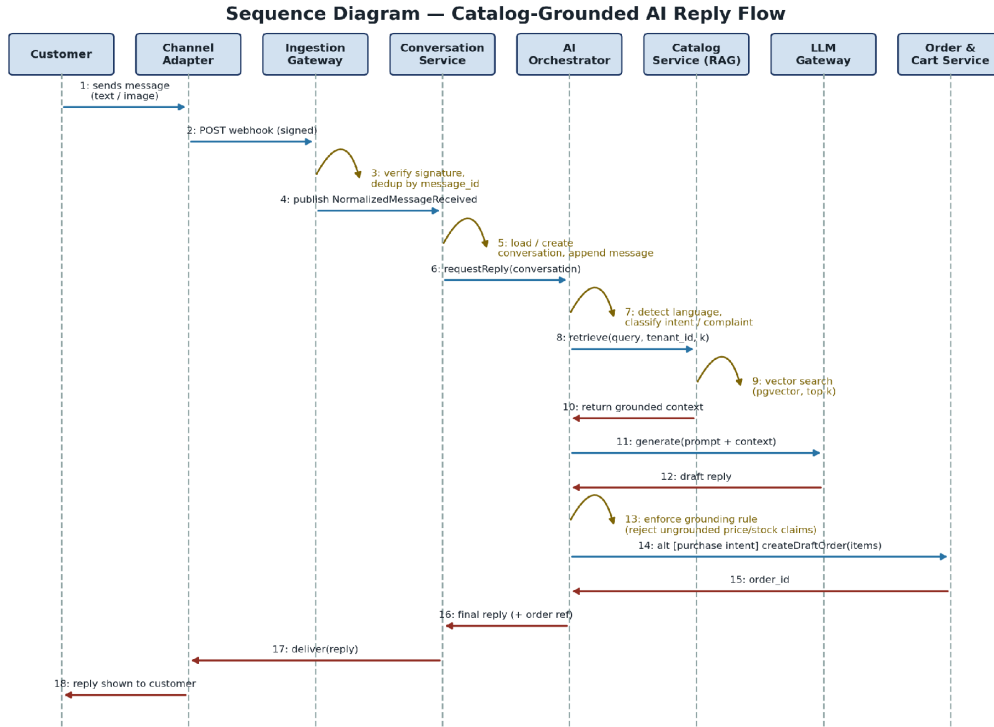


Figure 5.2 — Sequence Diagram for an Inbound Message

6. Relevant Equations and Algorithms

6.1 Semantic Retrieval Score (RAG)

Product-passage relevance is ranked by cosine similarity between the query embedding q and each candidate passage embedding d_i :

$$\text{sim}(q, d_i) = (q \cdot d_i) / (\|q\| \times \|d_i\|)$$

The retriever returns the top- k passages by this score (typically $k = 5$), and the Grounding Guardrail additionally requires $\text{sim}(q, d_i) \geq \tau$ for the top hit, where τ is a per-tenant confidence threshold; below τ the system prefers the safe fallback template over generating a possibly ungrounded answer.

6.2 Intent / Complaint Classification

The classifier produces a probability distribution over intent classes via softmax on its output logits z :

$$P(\text{class} = i \mid \text{msg}) = e^{z_i} / \sum_j e^{z_j}$$

A message is routed to the human complaint queue when $P(\text{complaint} \mid \text{msg}) \geq 0.5$; this threshold is intentionally lower than a typical 0.5-only “most-likely-class” rule for other intents, since the cost of missing

a genuine complaint (an unhappy, unaddressed customer) is judged higher than the cost of an occasional false-positive human escalation.

6.3 Grounded-Generation Constraint

For a generated reply r containing a set of factual claims $C(r)$ (prices, stock counts, delivery promises), the Guardrail enforces:

$$\forall c \in C(r): \exists d_i \in \text{retrieved}(q) \text{ such that } \text{supports}(d_i, c) = \text{true}$$

i.e., every factual claim in the reply must be traceable to at least one retrieved catalog passage; any claim that fails this check is stripped and the reply is regenerated with the offending span removed from the allowed output space.

6.4 Rate Limiting (Token Bucket)

Each tenant's outbound Delivery Worker is protected by a token-bucket limiter with capacity B and refill rate ρ tokens/second:

$$\text{tokens}(t) = \min(B, \text{tokens}(t - \Delta t) + \rho \cdot \Delta t); \text{ send allowed iff } \text{tokens}(t) \geq 1$$

This keeps every tenant's outbound send rate within the ceiling each channel's API imposes, and prevents one merchant's traffic burst from starving another tenant's delivery queue.

6.5 Usage Metering and Quota Check

A tenant's AI-message usage for billing period p is the count of `usage_events` of type `ai_reply` in that period, and a new AI generation is blocked once the quota is reached:

$$\begin{aligned} \text{usage}(\text{tenant}, p) &= |\{e \in \text{usage_events} : e.\text{tenant} = \text{tenant} \wedge e.\text{period} = p \wedge e.\text{type} = \text{ai_reply}\}| \\ \text{allow_ai_reply}(\text{tenant}) &= \text{usage}(\text{tenant}, \text{current_period}) < \text{subscriptions.message_quota} \end{aligned}$$

6.6 Reported Success Metrics

The Analytics Dashboard (Figure 2.1) surfaces three of the success metrics defined in the CO1 proposal (Section 3.5) computed directly from the schema in Section 4:

- **AI Automation Rate** = $(\text{messages with ai_generated} = \text{true}) / (\text{total outbound messages})$ over the reporting window.
- **Order Uplift** = $(\text{orders created in current window} - \text{orders in prior comparable window}) / \text{orders in prior window}$, isolating the incremental effect of AI-assisted checkout.
- **Average Response Time** = $\text{mean}(\text{first_outbound_message.created_at} - \text{conversation.created_at})$ across all conversations in the window.